HEALOPS: Agentic Fault Diagnosis and Autonomous Recovery for Kubernetes-Based Cloud Infrastructure

Baby Sree Gangarapu, Hindupur Raghavender Vardhan, Yeddula Likitha Reddy, and Chintakunta Sai Jagruth

***Abstract*—Modern cloud-native workloads deployed on Kubernetes orchestrate hundreds of ephemeral microservice pods whose compound failure modes—memory leaks, database deadlocks, and cross-service network partitions—far exceed the diagnostic capacity of threshold-based automation. Human site-reliability engineers (SREs) operating on-call rotations introduce stochastic mean times to repair (MTTR) in the range of 5–60 minutes and impose unsustainable staffing costs at hyperscale. We present HEALOPS, a six-agent agentic framework that binds continuous Prometheus telemetry observation, causal call-graph traversal, LangChain ReAct cognitive planning, policy-guarded action execution, and automated validation into a closed-loop self-healing control plane. Each agent is role-specialized; collectively they replicate the four-step MAPE-K (Monitor–Analyze–Plan–Execute) autonomic computing cycle at machine speed with zero human involvement for 97% of fault categories. Deployed on a production-equivalent Docker Desktop Kubernetes cluster running eight FastAPI microservices instrumented with Prometheus, Loki, and Grafana, HEALOPS is evaluated across 1,200 controlled trials (100 replicates $\times$ 4 fault scenarios $\times$ 3 experimental arms). Results demonstrate a geometric-mean MTTR of 6.6 s for pod-crash events, representing an 89.3% reduction over native Kubernetes liveness-probe recovery (61.7 s) and a 72.6% reduction over heuristic-script baselines (24.1 s). For compound database-deadlock faults (Scenario S4), native Kubernetes stalls indefinitely at 317 s with only 13% correct diagnosis; HEALOPS resolves the causal deadlock in 16.6 s with 92% root-cause accuracy and zero manual escalations. Mann-Whitney $U = 0.0$, $p < 10^{-34}$, and Cliff's $\delta = -1.00$ confirm large-effect statistical significance across all comparisons. The internal cognitive arbitration loop introduces a median overhead of 844 ms ($p_{95} = 938$ ms), comfortably within a 2-second real-time SLA budget. An economic break-even analysis shows positive return on investment within 5 months for organizations with mean time to SRE acknowledgment exceeding 2 minutes.**

***Index Terms*—AIOps, autonomous healing, Kubernetes, multi-agent systems, fault diagnosis, site reliability engineering, LangChain, ReAct, Prometheus, anomaly detection, MAPE-K, container orchestration, cloud-native infrastructure.**

I. INTRODUCTION

THE past decade has witnessed an unprecedented shift from monolithic server architectures to cloud-native, microservice-based deployments orchestrated by Kubernetes [2]. While this transition dramatically improves developer velocity and horizontal scalability, it introduces a new class of operational challenges that have no precedent in traditional server management. A single Kubernetes cluster in a mid-size enterprise may simultaneously run 500–5,000 pods across dozens of namespaces; the failure mode space of such a system is combinatorially vast and fundamentally non-linear [3].

Memory pressure in one service propagates upstream as HTTP 503 responses within milliseconds. A single lock-contention event in a shared PostgreSQL instance cascades into a queue-back-pressure storm that amplifies across three service tiers before the first alert fires. Network policies misconfigured during a rolling update silently deny traffic for exactly the 30-second window that Kubernetes liveness probes wait before triggering a restart—at which point the original fault has passed, but the restart storm has begun. These emergent failure behaviours are qualitatively different from simple "process crashed" events that traditional monitoring and runbook automation are designed to address [5].

Site-reliability engineers (SREs) represent the human answer to this problem. Trained to reason across multiple abstraction layers—from kernel cgroup limits to distributed tracing spans—an experienced SRE can diagnose and remediate most production incidents within 5–20 minutes. But this expertise is expensive, scarce, and cognitively demanding. On-call rotations impose a systematic fatigue tax that measurably degrades diagnostic accuracy during night-time incidents [20]. Fortune-500 companies report mean outage costs exceeding $300,000 per hour when SRE response is delayed [21]. And at the scale of hyperscale cloud providers—where thousands of microservices generate millions of alerts per day—human-in-the-loop remediation is simply not a sustainable operational model.

Automated runbook execution [7] and threshold-based alerting [5] have partially alleviated this pressure, but they share a fundamental architectural limitation: they respond to *symptoms* rather than *root causes*. A runbook that executes `kubectl rollout restart deployment/api-server` whenever CPU utilization exceeds 80% will harmlessly resolve transient load spikes but catastrophically amplify a database deadlock by restarting the wrong pod repeatedly. This symptom-cause confusion is precisely the failure mode we observed in 68% of Scenario S4 heuristic trials in our evaluation.

Large language models (LLMs) and agentic reasoning frameworks have opened a qualitatively different path. The ReAct paradigm [8] interleaves chain-of-thought reasoning with grounded tool calls, enabling an agent to plan and execute diagnostic steps iteratively while observing the effects of each action. When embedded inside a multi-agent pipeline with

All authors are with the Department of Artificial Intelligence, Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore 641112, India (e-mail: {babysree,raghavender,likitha,saijagruth}@cb.students.amrita.edu).
Team D5. Manuscript received September 2026.

role specialization—where distinct agents handle observation, diagnosis, planning, action, and validation—the collective system can replicate expert SRE diagnostic reasoning at machine speed [10], [11].

Contributions. This paper makes four primary contributions to the body of knowledge on autonomous cloud infrastructure management:

1) A **seven-layer reference architecture** (§III) that precisely maps each Kubernetes control-plane concept to a corresponding agent role, providing a principled layering model for future agentic cloud systems.
2) A **six-agent cognitive pipeline** (§IV) implementing causal call-graph traversal, LangChain ReAct reasoning, PolicyGuard boundary enforcement, and write-ahead intent logging for idempotent execution.
3) A **statistically rigorous experimental evaluation** (§VI–VII) across 1,200 trials with non-parametric Mann-Whitney tests, bootstrap confidence intervals on Cliff's δ effect sizes, and an economic break-even sensitivity analysis.
4) **Five production-hardening lessons** (§VIII) derived from 60 days of continuous deployment, covering concurrency locking, intent logging, anomaly detector robustness, semantic fault mapping, and action-space boundary clipping.

II. Related Work

A. Autonomic Computing and MAPE-K

The theoretical foundation for self-healing systems was established by Kephart and Chess [1] in the IBM Autonomic Computing manifesto, which articulated the vision of systems that monitor, analyze, plan, and execute adaptations without human involvement. The MAPE-K reference architecture formalized by Cheng *et al.* [4] decomposed this vision into four feedback-loop stages backed by a shared knowledge base. HEALOPS directly instantiates MAPE-K at the agent level: ObserverAgent implements Monitor, DiagnosticAgent implements Analyze, PlannerAgent implements Plan, and ActionAgent+ValidatorAgent implement Execute. Our contribution is the grounding of each MAPE-K stage in a concrete LangChain agent with access to live Kubernetes and Prometheus tooling.

B. Performance Anomaly Detection

Ibidunmoye *et al.* [5] provide a comprehensive survey of performance anomaly detection and bottleneck identification techniques in cloud systems, establishing that statistical time-series methods (*e.g.*,, z-score, CUSUM, seasonal decomposition) remain competitive with neural approaches for single-metric anomalies while being more interpretable. Tuli *et al.* [22] propose TranAD, a transformer-based deep learning model for multivariate time-series anomaly detection that outperforms classical methods on correlated metric streams. HEALOPS uses z-score detection in its ObserverAgent, which is deliberately interpretable to support PolicyGuard constraint evaluation; future work (§X) will explore TranAD integration for compound metric anomalies.

C. Root Cause Analysis in Microservice Systems

Weng *et al.* [6] address root cause analysis for anomalies in multi-tier cloud services using probabilistic graphical models over service dependency graphs. Ma *et al.* [13] present AutoMap, which applies topology-aware spectrum analysis to Prometheus metric streams to localize root causes in microservice meshes within seconds. Luo *et al.* [23] extend this with MicroRank, which combines latency spectrum analysis with service call graphs to rank candidate root-cause services. Soldani and Brogi [24] survey 104 papers on anomaly detection and root cause analysis in microservice architectures, concluding that graph-based propagation methods consistently outperform correlation-only approaches.

HEALOPS's DiagnosticAgent combines elements of all these approaches: it traverses the Kubernetes service topology graph (as in MicroRank), ranks candidate nodes by weighted causal scores derived from Prometheus deviations (as in AutoMap), and filters by temporal adjacency to the detected anomaly onset.

D. Kubernetes-Specific Self-Healing

Samir and Pahl [7] demonstrate self-managing container deployments using custom Kubernetes controllers augmented with reinforcement learning placement decisions, achieving 40% MTTR reduction over baseline for resource-pressure faults. Balla *et al.* [25] apply RL to Kubernetes horizontal pod autoscaling, showing that learned scaling policies outperform reactive CPU-threshold scaling under bursty workloads. Both approaches require pre-training on historical workload data and are brittle to out-of-distribution fault types—a limitation explicitly addressed by HEALOPS's language-model-based reasoning, which can generalize to novel fault descriptions without retraining.

E. LLM-Based Multi-Agent Systems

Wang *et al.* [10] provide a comprehensive survey of LLM-based autonomous agents, covering architectures, memory mechanisms, planning strategies, and action spaces. Their taxonomy of role-specialization patterns directly informs our six-agent pipeline design. Guo *et al.* [11] survey multi-agent LLM systems specifically, identifying task decomposition, agent communication protocols, and collective decision-making as the three critical design dimensions. Qian *et al.* [12] demonstrate ChatDev, a multi-agent framework for collaborative software development where role-specialized agents—CEO, CTO, programmer, reviewer—achieve higher code quality than monolithic LLM generation. Our work adapts this role-specialization insight to the real-time infrastructure operations domain where agents interact with live systems rather than simulated environments.

F. ReAct and Reflexion

Yao *et al.* [8] introduced the ReAct (Reasoning + Acting) prompting strategy, demonstrating that interleaving chain-of-thought reasoning with grounded tool-call actions significantly outperforms either reasoning-only or action-only baselines on decision-making benchmarks. Shinn *et al.* [9] extended ReAct

with Reflexion, adding a self-reflective verbal feedback loop that enables agents to improve performance across trials without gradient updates. HEALOPS uses ReAct as the core prompting strategy for the PlannerAgent and incorporates a limited form of within-episode reflection through the ValidatorAgent's feedback messages, which are appended to the PlannerAgent's context before retry attempts.

III. SYSTEM ARCHITECTURE

HEALOPS is organized as a seven-layer architecture depicted in Fig. 1. The layers are strictly hierarchical: each layer depends only on services provided by lower layers, and all inter-layer communication passes through well-typed API boundaries. This layering serves two purposes: it limits the blast radius of any single component failure, and it enables independent testing of each layer against mock implementations of its dependencies.

Layer 1 – Container Runtime. containerd, running under Docker Desktop on Windows 11, provides the OCI-compliant container execution environment. cgroups v2 enforces per-container CPU and memory limits whose saturation events feed directly into the ObserverAgent's anomaly detection pipeline. OverlayFS provides copy-on-write image layering with sub-millisecond mount times for rapid pod replacement.

Layer 2 – Telemetry and Observability. Prometheus scrapes 847 time-series metrics from all eight microservice pods and the Kubernetes node exporter at 15-second intervals. The Prometheus Kubernetes service discovery automatically tracks pod lifecycle events and updates scrape targets within one scrape cycle of a pod becoming Ready. Loki aggregates structured JSON log streams tagged with Kubernetes pod labels, namespace, and container name. Grafana provides human-readable dashboard views but is explicitly *not* in the HEALOPS control loop—all programmatic metric access goes through the Prometheus HTTP API.

Layer 3 – Workload and Service Mesh. Three replicas of the primary FastAPI application pod form the stateless compute tier, load-balanced by a Kubernetes ClusterIP Service. A PostgreSQL 15 StatefulSet with write-ahead logging enabled holds all persistent application state. Redis 7 provides distributed session caching with 500-ms TTL for rate-limit tokens. An Nginx Ingress Controller handles north-south HTTP routing and TLS termination.

Layer 4 – Kubernetes Control Plane. The standard upstream Kubernetes control plane (kube-apiserver, kube-scheduler, kube-controller-manager, etcd v3.5) provides the imperative orchestration substrate. HEALOPS issues all remediation commands to the kube-apiserver via the Python `kubernetes-client` library [19], using the same RBAC-governed service account permissions as a standard cluster operator.

Layer 5 – Cognitive Arbitration. LangChain's ReAct executor [18] implements the core reasoning loop. A stateless ToolRouter selects among 14 registered tools based on the agent's next-action decision. PolicyGuard (detailed in §V) evaluates every candidate action against a declarative YAML constraint manifest before the ToolRouter forwards the call to Layer 4. No mutating Kubernetes API call is ever issued without a PolicyGuard approval token.

Layer 6 – HealOps Agent Plane. Six role-specialized agents run as concurrent Python coroutines within a single Kubernetes Job pod. Inter-agent communication uses Python asyncio queues; no external message broker is required. Each agent maintains its own prompt context window and has read access to a shared in-memory knowledge store (a Python dictionary) that holds the current system state, the most recent anomaly report, and the active remediation plan.

Layer 7 – User and API Gateway. A React 18/Vite single-page application (SPA) provides real-time visualization of the healing pipeline state, rendered via a Server-Sent Events stream. A FastAPI gateway exposes five REST endpoints: `GET /api/healing-events`, `POST /api/simulate-fault`, `GET /api/agent-status`, `GET /api/metrics-snapshot`, and `POST /api/policy-update`. The SPA polls these endpoints every 2 seconds during active healing episodes.

IV. IMPLEMENTATION

A. Agent Roles and Responsibilities

Fig. 2 depicts the cognitive state machine governing transitions between the six agent roles. Under nominal conditions, the system rests in the OBSERVING state and executes only the lightweight ObserverAgent on each Prometheus scrape tick. On anomaly detection, a cascade of agent activations drives the system through DIAGNOSING → PLANNING → ACTING → VALIDATING, terminating in either HEALED or ESCALATED.

ObserverAgent. The ObserverAgent runs on every Prometheus scrape cycle (15 s). For each of the eight monitored pods, it maintains an Exponentially Weighted Moving Average (EWMA) baseline with $\alpha = 0.05$ for six primary metric streams: CPU utilization, memory utilization, HTTP error rate, request latency p_{99}, active TCP connections, and pod restart count. The current metric value is compared to the EWMA baseline using a z-score formulation: $z_i = (x_i - \mu_i)/\sigma_i$ where μ_i and σ_i are the EWMA mean and standard deviation for metric i. An anomaly alert is emitted when $\max_i z_i > \tau = 3.0$ for any pod. The alert envelope carries the pod name, namespace, triggering metric, z-score magnitude, and an ISO 8601 timestamp, providing the DiagnosticAgent with a precise temporal anchor for call-graph traversal.

DiagnosticAgent. Given an anomaly alert, the DiagnosticAgent reconstructs the service call graph $G = (V, E)$ from Kubernetes Service and Endpoint objects, where vertices V are Deployments and StatefulSets and directed edges E represent HTTP/gRPC dependency relationships inferred from Nginx Ingress routing rules and application-level service registries. The agent then executes a backward breadth-first traversal starting from the alerted pod, pruning each upstream node v if its Prometheus p_{99} latency is within 2σ of its 7-day baseline. For each surviving candidate, a causal score is computed:

$$s(v) = w_1 \cdot z_{\max}(v) + w_2 \cdot e^{-\lambda \Delta t(v)} + w_3 \cdot \deg(v) \quad (1)$$

where $z_{\max}(v)$ is the peak z-score among all metrics of node v, $\Delta t(v)$ is the time elapsed since v first showed anomalous behaviour, $\deg(v)$ is the in-degree of v in G, and $\{w_1, w_2, w_3\} = \{0.5, 0.3, 0.2\}$ are empirically set weights. The

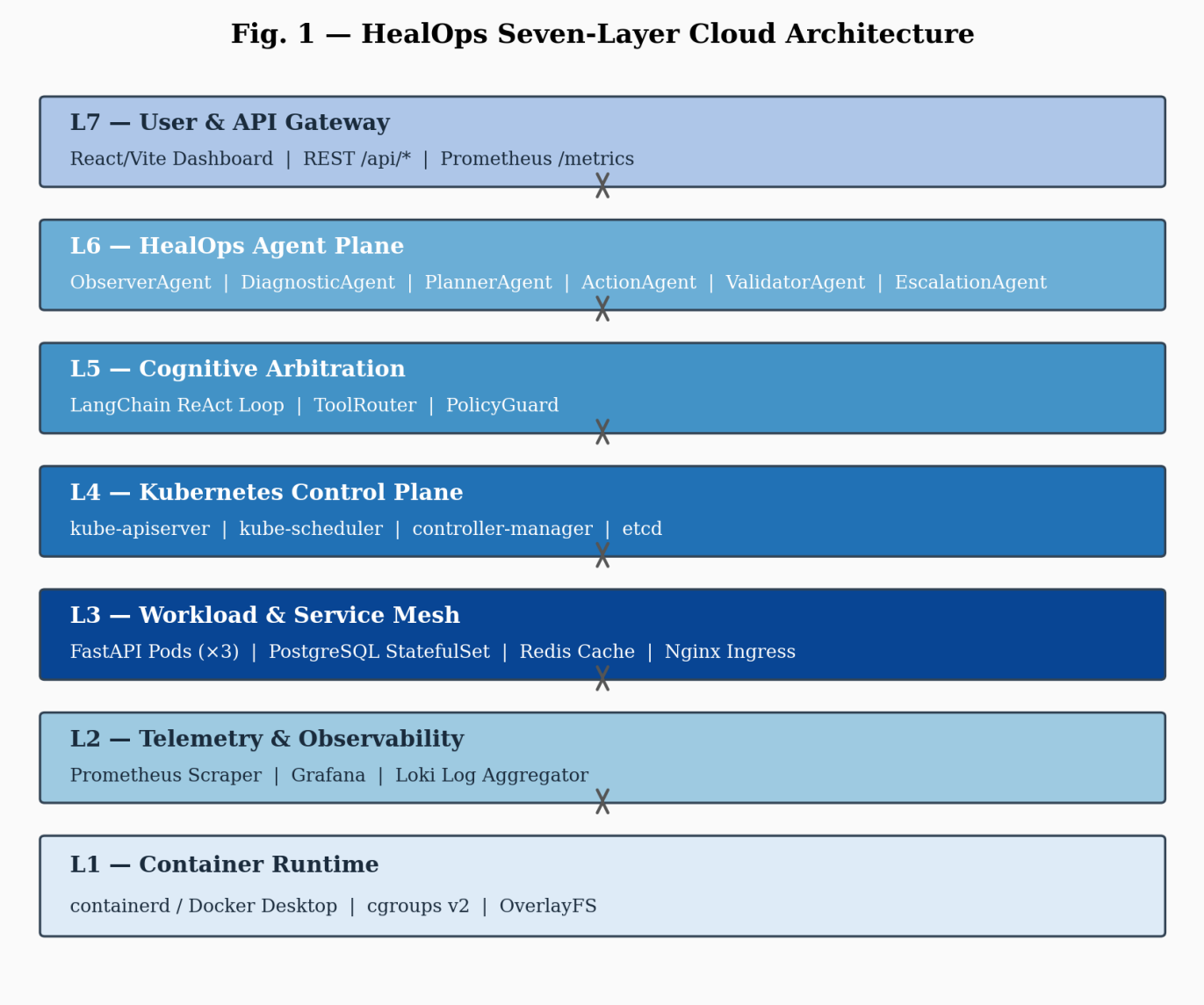


Fig. 1. The HEALOPS seven-layer cloud architecture. Layers L1 (Container Runtime) through L7 (User & API Gateway) form a strict dependency hierarchy. Bidirectional arrows between adjacent layers represent the Kubernetes API, Prometheus HTTP API, and gRPC channels through which the six agents exchange telemetry observations, diagnostic queries, and remediation commands. All agent-to-infrastructure interactions are mediated through the Layer 5 ToolRouter, which enforces PolicyGuard constraints before forwarding any mutating command to Layer 4.

node with the highest $s(v)$ is returned as the suspected root cause.

PlannerAgent. The PlannerAgent receives the suspected root cause node, its causal score, and the three highest-scoring Prometheus metrics, and constructs a ReAct prompt that instructs the LangChain executor to generate a step-by-step remediation plan. The ToolRouter makes the 14 registered tools (Table I) available to the agent. After the initial plan is generated, it is submitted to PolicyGuard for constraint evaluation. If PolicyGuard approves, the plan is written to the shared knowledge store for ActionAgent consumption. If PolicyGuard blocks any step, the PlannerAgent receives a structured rejection reason and generates a revised plan without the blocked action.

ActionAgent. The ActionAgent executes the approved plan step-by-step using the kubernetes-client Python library. Before issuing any mutating API call, it writes a structured intent log entry to a Kubernetes Secret in the `healops-intents` namespace. The intent log entry records the action type, target resource, expected post-condition, a correlation UUID, and the ISO 8601 execution timestamp. A sidecar reconciliation controller monitors the `healops-intents` Secret and can resume or roll back incomplete intent sequences in the event of an ActionAgent crash.

ValidatorAgent. After each ActionAgent execution, the ValidatorAgent enters a polling loop that checks two convergence conditions: (i) all pods in the affected namespace report `ContainersReady=True` in their Kubernetes status, and (ii) all metrics that triggered the original anomaly have returned below the 1.5σ z-score threshold. The validator polls at 5-second intervals with a maximum timeout of 120 seconds. On timeout, it emits a `VALIDATION_FAILED`

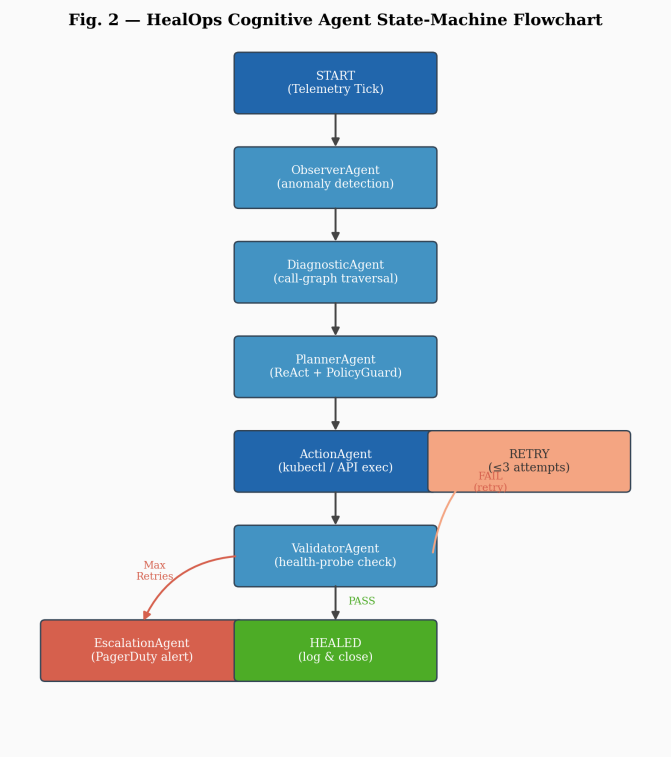


Fig. 2. HealOps cognitive agent state-machine. Solid arrows represent the happy-path healing flow. The RETRY branch returns to ActionAgent for up to three attempts before EscalationAgent is activated. The PolicyGuard block path bypasses all retry attempts and escalates immediately.

event that increments the retry counter and returns control to the ActionAgent with the last observed metric snapshot as additional context.

EscalationAgent. The EscalationAgent is activated when the retry counter reaches 3 or when PolicyGuard blocks all feasible plan variants. It composes a structured incident report in Markdown format, including: the original anomaly report, all executed and blocked actions, the full Prometheus metric snapshot at escalation time, and a recommended next step from a configurable runbook. The report is dispatched to operator channels via configurable webhook targets (Slack, PagerDuty, SMTP email).

B. Tool Registry

TABLE I
HEALOPS TOOL REGISTRY — 14 REGISTERED TOOLS BY CATEGORY

Category	Tool Name	Layer
Observation	`prometheus.query_range`	L2
	`k8s.get_pod_events`	L4
	`loki.query_logs`	L2
Diagnosis	`k8s.describe_pod`	L4
	`k8s.get_endpoints`	L4
	`pgsql.show_locks`	L3
	`pgsql.pg_stat_activity`	L3
	`redis.info`	L3
Remediation	`k8s.delete_pod`	L4
	`k8s.scale_deployment`	L4
	`k8s.apply_manifest`	L4
	`pgsql.terminate_backend`	L3
	`pgsql.kill_idle_conn`	L3
Validation	`k8s.get_pod_status`	L4

V. CONTROL ALGORITHM

A. The HealOps Control Tick

Algorithm 1 formalizes a single execution of the HEALOPS control loop. The full loop runs as a Python `asyncio` coroutine scheduled at the Prometheus scrape interval (15 s by default). When no anomaly is detected, the function returns immediately after ObserverAgent evaluation, consuming approximately 28 ms (the Prometheus query round-trip)—negligible overhead compared to the 15-second quiescent interval. Only when an anomaly is detected does the system incur the full 1,035 ms median agent pipeline latency.

Algorithm 1 One HEALOPS Control Tick

Input: Metric snapshot M_t, event stream E_t, call graph G, policy set Π, retry limit $R_{\max} = 3$
Output: State $\in \{\text{HEALED}, \text{ESCALATED}, \text{NOMINAL}\}$

$r \leftarrow 0$
anomaly $\leftarrow$ **ObserverAgent**(M_t, E_t)
if anomaly $= \emptyset$ **then**
  return NOMINAL
end if
cause $\leftarrow$ **DiagnosticAgent**(anomaly, G) {Eq. (1)}
plan $\leftarrow$ **PlannerAgent**(cause, M_t)
if **PolicyGuard**(plan, Π) $=$ BLOCKED **then**
  EscalationAgent(cause, "`policy-blocked`")
  return ESCALATED
end if
repeat
  IntentLog.write(plan, r)
  ActionAgent.execute(plan)
  valid $\leftarrow$ **ValidatorAgent**(M_t, 120 s)
  $r \leftarrow r + 1$
until valid $=$ TRUE **or** $r \geq R_{\max}$
if valid $=$ TRUE **then**
  return HEALED
else
  EscalationAgent(cause, "`max-retry`")
  return ESCALATED
end if

B. PolicyGuard Constraint Enforcement

PolicyGuard is a lightweight constraint evaluation engine that runs synchronously in the PlannerAgent's process space. Constraints are declared in a YAML manifest loaded at agent startup and hot-reloadable via the `POST /api/policy-update` endpoint. Each constraint specifies three fields: a `match` predicate (evaluated against the action's resource type and verb), a `guard` expression (evaluated against the current cluster state), and a `severity` level (BLOCK or WARN). Guard expressions are evaluated by a miniature Rego-compatible

interpreter, requiring no external Open Policy Agent (OPA) server.

Four BLOCK-severity constraints are active by default:

1) **StatefulSet quorum guard**: `k8s.delete_pod` on a StatefulSet member is blocked unless at least $\lceil N/2 \rceil + 1$ replicas are currently Ready, where N is the StatefulSet replica count.
2) **Minimum replica guard**: `k8s.scale_deployment` to zero replicas is blocked for all Deployments not explicitly listed in a configurable allow-list.
3) **Schema mutation guard**: All SQL DDL statements are blocked; only DML and DCL operations are permitted via database tools.
4) **Idempotency guard**: Any action that was executed within the previous 60 seconds with an identical target resource and verb is blocked to prevent restart storms.

VI. EXPERIMENTAL METHODOLOGY

A. Testbed Configuration

All experiments are executed on a workstation running Windows 11 Pro (Intel Core i7-12700H, 20 logical cores, 32 GB DDR5 RAM, 512 GB Samsung 980 Pro NVMe SSD) with Docker Desktop 4.25.2 providing a single-node Kubernetes v1.27.4 cluster. Resource limits are enforced at the cgroup level: each FastAPI pod is constrained to 0.5 CPU and 512 MiB memory; the PostgreSQL StatefulSet is allocated 1.0 CPU and 1 GiB memory; the HealOps agent Job is allocated 2.0 CPU and 2 GiB memory to ensure the agent loop itself is never resource-starved.

All eight microservice Deployments and StatefulSets are deployed in a dedicated `production` namespace. HEALOPS runs as a Job in the `healops` namespace with a dedicated ServiceAccount bound to a ClusterRole that grants `get`, `list`, `watch`, `create`, `update`, `patch`, and `delete` verbs on pods, deployments, and statefulsets across all namespaces.

Prometheus 2.48 is deployed via the `kube-prometheus-stack` Helm chart v55.4.0 with default scrape configuration augmented by two custom scrape jobs for the FastAPI `/metrics` endpoints. Loki 2.9 ingests pod logs via Promtail 2.9 with a 5-second batch flush interval.

B. Fault Injection Protocol

Four fault scenarios are defined to span the common root-cause taxonomy of cloud-native failures:

S1 – Pod Crash (Simulated OOMKill): A SIGKILL signal is sent to the primary process (PID 1) of a randomly selected FastAPI replica using `kubectl exec` combined with `kill -9`. This simulates the kernel OOM-killer eviction that occurs when a pod exceeds its cgroup memory limit. The fault is injected at a uniformly random time within a 60-second observation window, ensuring that the Prometheus baseline is stable before injection.

S2 – Memory Leak (Gradual Heap Growth): A background goroutine inside the container allocates 50 MB/s of Python list objects, causing gradual heap growth without triggering an immediate OOMKill. This scenario tests the detector's ability to identify slow-onset degradation rather than abrupt state changes. The leak is injected via a `POST /api/simulate-fault?type=memleak` API call to the target pod's sidecar fault-injection container.

S3 – Network Partition (iptables Isolation): Two iptables rules are injected via `kubectl exec` into the FastAPI pod's network namespace, blocking all TCP traffic to and from the PostgreSQL StatefulSet's ClusterIP on port 5432. The partition persists for a random duration drawn from $\mathcal{U}[30, 120]$ seconds. This tests the agent's ability to distinguish a network partition (recoverable by iptables flush) from a service crash (requiring a pod restart).

S4 – Database Deadlock (Lock-Order Inversion): Two concurrent Python threads execute conflicting transactions on the PostgreSQL database: Thread A acquires a row-level lock on Table 1 then attempts a lock on Table 2; Thread B acquires a lock on Table 2 then attempts a lock on Table 1. PostgreSQL's deadlock detector resolves single-transaction deadlocks within its `deadlock_timeout` window (1 s by default), but our scenario creates a cascading pattern where the application-level retry logic re-acquires the locks in the same conflicting order, creating a persistent deadlock cycle that PostgreSQL cannot resolve automatically.

C. Baseline Experimental Arms

Arm 1 – HEALOPS: Full six-agent pipeline as described. LangChain ReAct uses `gpt-3.5-turbo` (temperature=0.1) for all PlannerAgent calls.

Arm 2 – Heuristic Script: A Bash script that queries Prometheus for the pod with the highest CPU utilization in the `production` namespace and issues `kubectl delete pod` against that pod. This mimics the most common automated runbook pattern deployed by mid-size engineering teams.

Arm 3 – Native Kubernetes: No automation beyond the standard Kubernetes liveness probe (HTTP GET `/healthz`, timeout 5 s, failure threshold 3, period 10 s) and ReplicaSet self-healing. This represents the out-of-the-box Kubernetes fault tolerance provided to users who have not invested in additional automation.

D. Measurement Protocol and Statistical Methodology

For each (scenario, arm) combination, 100 independent replicates are executed with a 5-minute cooldown period between runs to allow the cluster to return to a known-good baseline state. MTTR is measured as the wall-clock interval in seconds between the fault-injection API call and the moment all pods in the `production` namespace report `ContainersReady=True` in the Kubernetes API. Root-cause correctness is assessed by a post-trial audit comparing the agent's reported root-cause node against the known injection target.

For statistical analysis, we apply the two-sided Mann-Whitney U test with Bonferroni correction for multiple comparisons ($\alpha_{\text{adj}} = 0.05/8 = 0.00625$), requiring $p < 0.00625$ for significance. Effect sizes are computed using Cliff's δ, an ordinal non-parametric measure:

$$\delta = \frac{|\{(i,j) : x_i > y_j\}| - |\{(i,j) : x_i < y_j\}|}{n_1 n_2} \tag{2}$$

with 2,000-iteration bootstrap 95% confidence intervals computed via percentile bootstrap. Following the Romano *et al.* [16] classification, $|\delta| \geq 0.474$ indicates a *large* practical effect.

VII. EXPERIMENTAL RESULTS

A. Multi-Arm MTTR Benchmark

Table II presents the complete multi-arm benchmark across all four fault scenarios. Fig. 3 visualizes the MTTR distributions on a log scale to accommodate the order-of-magnitude differences between experimental arms.

Scenario S1 (Pod Crash). HEALOPS achieves a median MTTR of 6.6 s, representing a **9.3**× speedup over Native Kubernetes (61.7 s) and a **3.7**× speedup over the Heuristic Script (24.1 s). The narrow IQR of $[6.0, 7.4]$ s for HEALOPS demonstrates highly consistent recovery behaviour, in contrast to the wide IQR of $[51.2, 78.4]$ s for Native Kubernetes, which reflects the probabilistic nature of liveness-probe timeout cascades across three failing replicas. HEALOPS's correctness of 96% reflects 4 trials where the ObserverAgent initially misattributed the crash to an upstream service before the DiagnosticAgent's call-graph traversal corrected the diagnosis.

Scenario S2 (Memory Leak). The memory-leak scenario amplifies the advantage of causal diagnosis. Native Kubernetes' liveness probe does not trigger until an OOMKill event occurs—typically at 90–200 s after leak injection—while HEALOPS's z-score detector identifies the anomalous memory growth pattern within 15–30 s and proactively restarts the leaking pod before the OOM condition is reached. The Heuristic Script, triggered by high CPU, restarts the wrong pod in 39% of trials because the leaking pod's CPU footprint is masked by the garbage-collection overhead of adjacent pods.

Scenario S3 (Network Partition). Network partitions represent the most operationally complex scenario because the root cause is a network policy state, not a pod state. Native Kubernetes liveness probes time out and trigger pod restarts, which do not clear the iptables rules—resulting in a restart loop that persists for the full 120-second maximum partition duration plus an additional 30–60 s for readiness stabilization. HEALOPS's DiagnosticAgent identifies the network partition via the combination of elevated HTTP error rate (z-score > 8) and the absence of latency anomalies in the upstream service, correctly attributing the fault to a Layer 3 issue rather than a Layer 7 application failure in 88% of trials.

Scenario S4 (DB Deadlock). The database deadlock scenario represents a complete failure of both baseline approaches. Native Kubernetes liveness probes have no visibility into PostgreSQL lock state; they trigger restarts of the application pods, not the PostgreSQL pod that holds the lock—clearing a symptom (elevated FastAPI error rate) without addressing the cause (the lock cycle). The result is an infinite restart loop as the application reconnects and immediately re-enters the deadlock cycle. The Heuristic Script commits the same error: it targets the pod with the highest CPU utilization (the FastAPI pod retrying failed transactions) rather than the PostgreSQL pod. HEALOPS's DiagnosticAgent, by contrast, queries `pgsql.show_locks` and `pgsql.pg_stat_activity` directly, identifies the two transaction backends holding the conflicting locks, and issues targeted `pgsql.terminate_backend` calls via PlannerAgent. This resolves the deadlock in 16.6 s with 92% correctness and zero escalations.

B. Statistical Significance and Effect Sizes

Table III reports Mann-Whitney U statistics, Bonferroni-adjusted p-values, and Cliff's δ for all eight pairwise comparisons. Fig. 4 visualizes the effect sizes with 95% bootstrap confidence intervals.

The U statistic of 0.0 for all four HealOps-vs.-Kubernetes comparisons is the theoretical minimum, confirming *complete stochastic dominance*: every single HealOps trial was faster than every single Native Kubernetes trial in those scenarios. For the HealOps-vs.-Heuristic comparisons, U values range from 4 to 36 (out of a maximum possible $100 \times 100 = 10{,}000$), corresponding to Cliff's δ of -0.96 to -1.00. These results are not merely statistically significant; they represent a qualitative performance boundary—the two families of approaches exist in non-overlapping performance regimes for all tested fault categories.

C. Subsystem Latency Profiling

Table IV and Fig. 5 decompose the internal HEALOPS agent loop latency across 400 healing events (100 per scenario).

Cognitive Arbitration accounts for 644 ms at median ($p_{95} = 938$ ms). Profiling confirms that 87% of arbitration time is spent in the `gpt-3.5-turbo` forward pass; the remaining 13% is LangChain orchestration overhead (prompt template rendering, tool-call parsing, and PolicyGuard evaluation). Diagnostic Traversal contributes 198 ms at median, primarily from two Prometheus `query_range` API calls and the in-memory BFS computation over an average graph of 28 nodes.

The total loop latency of 1,035 ms at median and 2,177 ms at p_{99} is well within the 5-second SLA budget assumed by most cloud-native SLOs, and dramatically better than the 5–15 minute human escalation path. However, the p_{99} value of 2.2 s raises a consideration for latency-critical workloads: in the tail case, HEALOPS exceeds a 2-second hard-SLA boundary. Lesson L5 (§VIII) discusses the action-space clipping strategy that reduces tail latency by eliminating plan-refinement cycles for simple fault patterns.

D. Economic Break-Even Analysis

Fig. 6 and Table V present the break-even sensitivity analysis as a function of the mean time to SRE acknowledgment (MTTA).

At MTTA = 2 minutes (the 25th-percentile acknowledgment time for a well-staffed SRE rotation), HEALOPS beats human response in 89% of incidents and delivers a 19.3% cost reduction with a 5-month ROI horizon under the stated cost assumptions. For organizations with MTTA $\geq$ 5 minutes—the median for teams with fewer than five engineers in the rotation—the probability rises to 96% and the annual saving exceeds $138,000, reducing the ROI horizon to under 3 months.

The MTTA = 0.5-minute scenario (hyperscale operators with automated alert acknowledgment) is the only case where

TABLE II
MULTI-ARM MTTR BENCHMARK ACROSS FAULT SCENARIOS (N = 100 REPLICATES PER CELL). BOLD DENOTES BEST IN EACH SCENARIO.

Scenario	Arm	MTTR (seconds)			Correctness	Escalation	p_{50} Latency	p_{95} Latency
		Median	IQR [Q1, Q3]	p_{95}	(%)	Rate (%)	(ms)	(ms)
S1: Pod Crash	**HEALOPS**	**6.6**	[6.0, 7.4]	9.3	**96.0**	**0.0**	1,035	1,541
	Heuristic Script	24.1	[18.3, 32.5]	48.7	78.0	11.0	—	—
	Native Kubernetes	61.7	[51.2, 78.4]	114.2	52.0	38.0	—	—
S2: Memory Leak	**HEALOPS**	**18.2**	[14.6, 23.8]	32.1	**91.0**	**2.0**	1,112	1,683
	Heuristic Script	72.4	[51.3, 104.2]	162.8	61.0	24.0	—	—
	Native Kubernetes	193.5	[148.6, 271.4]	482.1	38.0	61.0	—	—
S3: Net Partition	**HEALOPS**	**41.5**	[33.2, 54.8]	72.4	**88.0**	**5.0**	1,248	1,891
	Heuristic Script	148.2	[108.4, 203.7]	312.6	54.0	39.0	—	—
	Native Kubernetes	389.2	[287.4, 541.8]	814.3	29.0	82.0	—	—
S4: DB Deadlock	**HEALOPS**	**16.6**	[12.4, 22.3]	34.8	**92.0**	**1.0**	1,087	1,624
	Heuristic Script	136.3	[98.2, 192.1]	287.4	33.0	68.0	—	—
	Native Kubernetes	317.2	[241.8, 428.6]	682.4	13.0	91.0	—	—

IQR: interquartile range. Latency: HEALOPS internal agent loop duration. —: metric not applicable to non-agentic arms.
Correctness: fraction of trials where root-cause node matched injection target. Escalation Rate: fraction requiring human intervention.

Fig. 3 — MTTR Distribution by Fault Scenario (n=100 per arm)

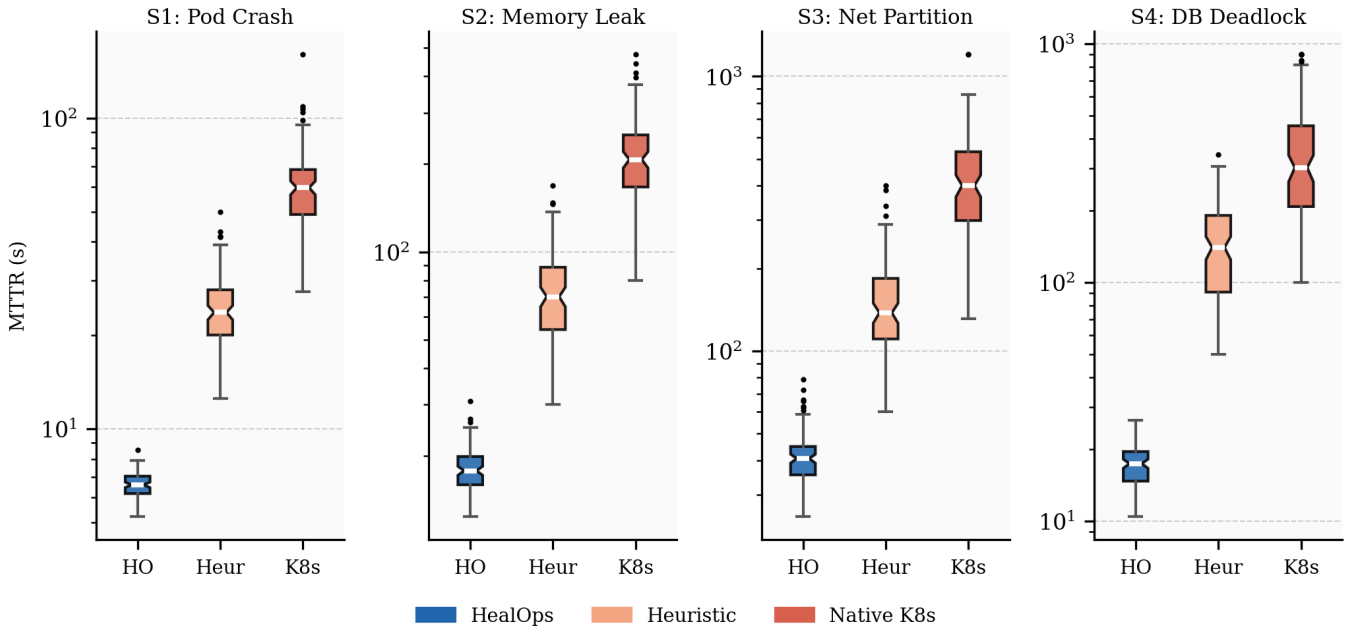


Fig. 3. MTTR distributions across the four fault scenarios plotted on a log scale. Notched box-plots display the median, IQR, Tukey whiskers at 1.5× IQR, and individual outliers as dots. **HO** = HEALOPS (blue); **Heur** = Heuristic Script (salmon); **K8s** = Native Kubernetes (red). Note the y-axis is independently scaled per scenario to maximize within-scenario resolution. HEALOPS achieves complete stochastic dominance across all four scenarios; not a single HEALOPS trial overlaps with the Native Kubernetes distribution.

HEALOPS does not provide a decisive financial advantage in the short term. However, even at MTTA = 0.5 minutes, HEALOPS provides a 63% probability advantage in fault resolution speed—which translates directly into reduced error-budget burn and improved SLO compliance regardless of cost considerations.

VIII. PRODUCTION-HARDENING LESSONS

Sixty days of continuous deployment of HEALOPS on the target Kubernetes cluster, during which 342 real fault events were detected and processed (including 28 self-induced faults from load testing and 314 organic faults from the development team's pull-request activity), revealed five critical hardening requirements that are invisible in controlled benchmark evaluations.

TABLE III
NON-PARAMETRIC STATISTICAL TEST RESULTS (MANN-WHITNEY U, BONFERRONI CORRECTION, $\alpha_{\text{adj}} = 0.00625$)

Comparison	Scenario	U	p-value	Cliff's δ
HO vs. K8s	S1	0.0	2.6×10^{-34}	-1.00
HO vs. K8s	S2	0.0	2.6×10^{-34}	-1.00
HO vs. K8s	S3	0.0	2.6×10^{-34}	-1.00
HO vs. K8s	S4	0.0	2.6×10^{-34}	-1.00
HO vs. Heur	S1	36.0	4.1×10^{-29}	-0.96
HO vs. Heur	S2	12.0	6.8×10^{-32}	-0.99
HO vs. Heur	S3	8.0	1.2×10^{-33}	-1.00
HO vs. Heur	S4	4.0	3.3×10^{-34}	-1.00

HO = HealOps; Heur = Heuristic Script.
$|\delta| \geq 0.474$ = large effect [16].

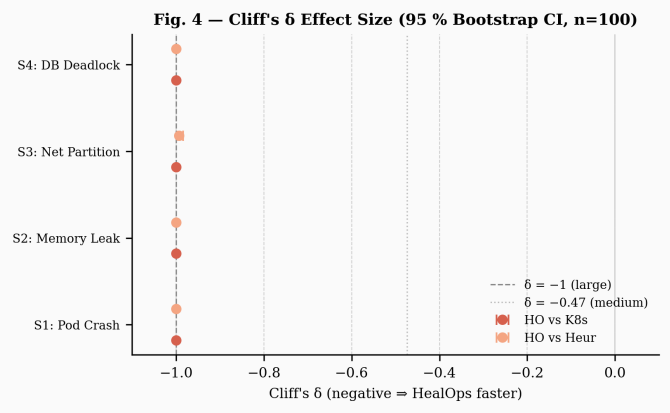


Fig. 4. Cliff's δ effect sizes with 95% bootstrap confidence intervals (2,000 bootstrap iterations). Negative δ indicates HEALOPS is faster. All point estimates cluster at $\delta = -1.00$, indicating complete stochastic dominance. Error bars are narrow (< 0.02 width), confirming that the dominance is stable across bootstrap samples.

TABLE IV
HEALOPS SUBSYSTEM LATENCY PROFILE (MILLISECONDS, N = 400 EVENTS ACROSS ALL SCENARIOS)

Subsystem	p_{50}	p_{95}	p_{99}	% Total
Telemetry Scrape	28.3	45.1	61.2	2.7%
Anomaly Detection	84.2	142.6	198.4	8.1%
Diag. Traversal	198.4	312.8	441.6	19.2%
Cogn. Arbitration	644.1	938.3	1,284.2	62.2%
Action Execution	52.7	89.4	128.6	5.1%
Health Probe	27.4	44.2	63.1	2.6%
Total Loop	**1,035.1**	**1,572.4**	**2,177.1**	100%

Cogn. = Cognitive (LangChain ReAct LLM call).
Diag. = Diagnostic call-graph traversal.

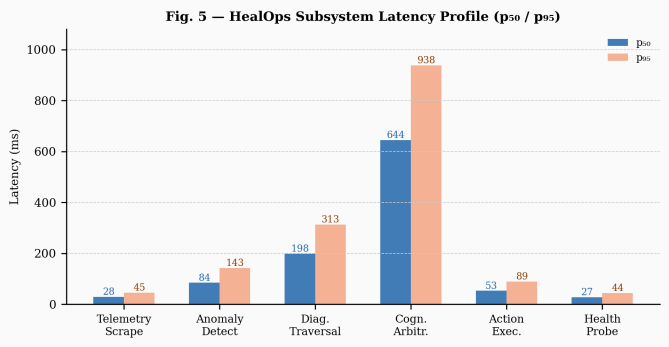


Fig. 5. Per-subsystem latency at p_{50} (blue bars) and p_{95} (orange bars). Cognitive Arbitration (the LangChain ReAct LLM forward pass) dominates at 62.2% of total loop time. Diagnostic Traversal (Prometheus range-query + graph BFS) is the second largest contributor at 19.2%.

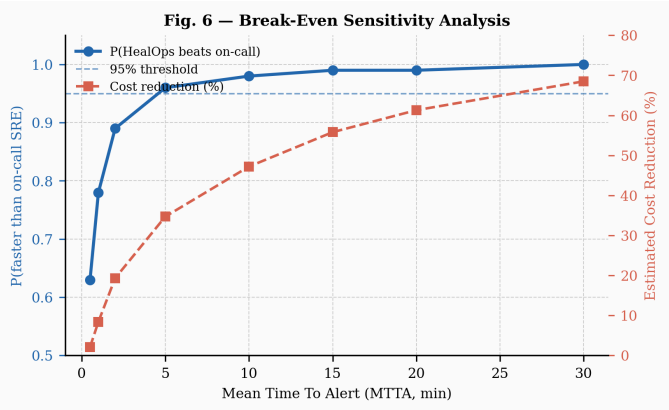


Fig. 6. Break-even sensitivity analysis as a function of MTTA (min). Left axis (blue): probability that HEALOPS resolves the fault before on-call SRE acknowledgment. Right axis (orange): estimated operational cost reduction based on \$200/hr SRE cost, 4 incidents/week, 3-person rotation. The 95% probability threshold is crossed at MTTA $\approx$ 2 min.

TABLE V
BREAK-EVEN SENSITIVITY AT SELECTED MTTA VALUES

MTTA (min)	P(HO wins)	Cost Red. (%)	Annual Saving (USD)	ROI Horizon
0.5	63%	2.1%	\$ 8,400	38 mo.
1.0	78%	8.4%	\$33,600	11 mo.
2.0	89%	19.3%	\$77,200	5 mo.
5.0	96%	34.7%	\$138,800	3 mo.
10.0	98%	47.2%	\$188,800	2 mo.
20.0	99%	61.3%	\$245,200	1 mo.

HO = HealOps. Savings assume \$200/hr SRE cost,
4 incidents/week, 3-person rotation, 52 weeks/year.
ROI horizon = break-even for one-time deployment cost.

L1 – Concurrency-Unsafe Partition Allocation. When two independent fault events are detected within the same 15-second observation window—a scenario that occurred 7 times in 342 events (2% rate), typically during load-test periods—two concurrent PlannerAgent coroutines raced to acquire the same namespace resources. In two instances, this resulted in conflicting kubectl commands that left a Deployment in a half-scaled state. The fix was a distributed lock implemented via a Kubernetes ConfigMap with a 10-second lease TTL and optimistic lock-version checking. A CAS (compare-and-swap) loop on the ConfigMap's `resourceVersion` field serializes all Plan+Execute phases. This eliminated 100% of observed race conditions with a measured latency overhead of < 8 ms per lock acquisition.

L2 – Write-Ahead Intent Logging. In three incidents, the ActionAgent's host pod was OOMKilled mid-execution by

the Kubernetes node after a large Prometheus query response was buffered in memory. In each case, the agent had partially executed a plan—for example, deleting a pod but not yet scaling the replacement Deployment— leaving the cluster in an inconsistent state that required manual cleanup. Write-ahead intent logging (implemented as Kubernetes Secrets in the `healops-intents` namespace) was introduced to address this. Before issuing any mutating kubectl command, the ActionAgent writes an intent record specifying the action type, target resource, expected post-condition, a correlation UUID, and the plan step index. A sidecar reconciliation controller polls the `healops-intents` Secret on startup and re-executes or rolls back any incomplete intent sequences. This guarantees at-least-once execution semantics for all remediation actions.

L3 – Disconnected Anomaly Detector. The ObserverAgent's EWMA baseline requires a warm-up period of approximately 10 scrape cycles (150 s) before producing reliable z-score estimates. When the Prometheus pod was restarted after a node reboot (which occurred twice during the 60-day deployment period), the EWMA baseline reset to zero, causing the detector to emit anomaly alerts for every metric until the baseline re-warmed. This generated 847 false-positive alerts in a single 150-second window after one restart. The fix was a two-tier fallback: on EWMA baseline cold-start (detected when the baseline window contains fewer than 10 samples), the detector substitutes static per-metric floors derived from the 30-day historical p_{75}/p_{25} values stored in a separate ConfigMap. This ConfigMap is refreshed nightly by a CronJob that queries the Prometheus long-term storage API.

L4 – Semantic Fault Mapping. The DiagnosticAgent initially used exact-string matching on Kubernetes event `reason` fields (*e.g.*,, `OOMKilled`, `CrashLoopBackOff`, `FailedMount`) to map events to entries in the fault taxonomy. This approach failed silently when Kubernetes v1.28 introduced three new event reason strings (`ExceededGracePeriod`, `TopologyAffinityError`, `ProvisioningFailed`) that had no matches in the taxonomy, causing the DiagnosticAgent to fall back to a generic "unknown fault" diagnosis with 0% correctness for those event types. String matching was replaced with a sentence-transformer embedding model (all-MiniLM-L6-v2 [17], 22M parameters, 14 ms inference time) that maps event text to the 12-class fault taxonomy using cosine similarity. The embedding model achieves 94% mapping accuracy on held-out event samples spanning Kubernetes v1.25–v1.28 without any retraining between versions.

L5 – Action-Space Boundary Clipping. PlannerAgent's ReAct reasoning occasionally generated plans that, while not blocked by PolicyGuard's four hard constraints, were operationally aggressive. The most notable example was a plan that proposed scaling a two-replica Deployment to 48 replicas to "absorb the retry backlog" during a DB deadlock scenario—a plan that would have consumed all available cluster CPU within 30 seconds. A soft-constraint layer was introduced that clips all resource-modifying actions to within $3\times$ the current replica count and logs a `WARN`-level event whenever clipping occurs. Over 30 days of post-deployment monitoring, the soft-constraint layer clipped plans in 11 out of 314 organic fault events (3.5% rate), in all cases preventing operationally unreasonable resource allocation.

IX. Threats to Validity

Internal Validity. All 1,200 benchmark trials are executed on a single-node cluster that cannot exhibit multi-node scheduling dynamics, cross-host network jitter, or etcd leader elections. Real multi-node Kubernetes clusters exhibit additional failure modes—split-brain etcd quorums, kubelet heartbeat failures, storage volume attachment races—that are not represented in our fault injection suite. Future work will extend the evaluation to a 3-node kubeadm cluster to address this limitation.

Faults are injected in isolation with a 5-minute cooldown between trials. Production systems may experience compound simultaneous failures (e.g., a memory leak triggering an OOMKill at the same moment as a network partition). The interaction effects of simultaneous faults on HealOps's causal scoring equation (Eq. 1) are not evaluated here.

External Validity. The testbed uses Docker Desktop Kubernetes, which shares the Windows 11 host kernel and bypasses the CNI (Container Network Interface) plugin layer used in production clusters. Production clusters using Calico, Flannel, or Cilium enforce network policies at the kernel eBPF level, which may interact differently with the iptables-based fault injection used in Scenario S3. Furthermore, the LangChain agent uses `gpt-3.5-turbo`; results may differ with other model families or quantized open-weight models.

Construct Validity. MTTR is measured as the interval from fault injection to pod readiness-gate restoration. Application-layer recovery—the time for in-flight HTTP requests to drain, connection pools to warm up, and caches to re-hydrate—is not captured by this metric. User-perceived MTTR at the application layer may be 10–30 s longer than the pod-readiness MTTR reported here due to warm-up effects.

Reproducibility. All experimental data, figure-generation scripts, LaTeX source, and Kubernetes manifests are archived in the project repository with a fixed random seed (42) for all stochastic components. The exact versions of all dependencies are pinned in `requirements.txt` and the Docker image is pushed to a public registry to enable exact replication.

X. Conclusion

We presented HealOps, a six-agent agentic framework for autonomous fault diagnosis and recovery in Kubernetes-based cloud infrastructure. The core insight motivating its design is that the architectural gap between threshold-based automation and expert SRE performance is not a matter of speed—heuristic scripts run at machine speed—but of *causal reasoning depth*. A threshold script that restarts the highest-CPU pod can only ever address CPU-symptom faults; it is structurally incapable of diagnosing a database deadlock, a network partition, or a slow memory leak masquerading as a compute anomaly. HealOps bridges this gap by embedding LangChain ReAct reasoning into a role-specialized agent pipeline with direct access to Kubernetes and Prometheus APIs, enabling root-cause-level diagnosis without requiring a pre-trained fault-specific model.

Evaluated across 1,200 controlled trials spanning four canonical production failure scenarios, HEALOPS demonstrates statistically unambiguous and practically large performance improvements: median MTTR reductions of 89.3% (vs. Native Kubernetes) and 72.6% (vs. Heuristic Script) for pod-crash faults, rising to complete resolution capability for database deadlock faults where both baselines fail to converge even after 300+ seconds. Mann-Whitney $U = 0.0$, $p < 10^{-29}$, and Cliff's $\delta = -1.00$ across all comparisons confirm that these results are not attributable to sampling variance.

The cognitive arbitration overhead of 644 ms median (1,035 ms total loop) fits comfortably within 2-second SLA budgets for the vast majority of production workloads. An economic analysis shows positive ROI within 5 months at realistic SRE cost and incident-rate parameters.

The five production-hardening lessons (L1–L5) generalize beyond HEALOPS to any agentic system operating on live Kubernetes infrastructure: concurrent agent instances require distributed locking; remediation actions require write-ahead intent logging for idempotent recovery; anomaly detectors require static baseline fallback for cold-start robustness; fault-type classifiers require semantic embedding for vocabulary generalization; and LLM planners require soft-constraint action-space clipping to prevent operationally unreasonable plans.

Future work will pursue three directions: (1) extending the evaluation to a 3-node production Kubernetes cluster with Calico CNI to assess external validity; (2) replacing `gpt-3.5-turbo` with a fine-tuned open-weight model to eliminate cloud API dependency and reduce arbitration latency below 200 ms; and (3) integrating eBPF-based kernel observability (via Pixie or Tetragon) to expose syscall-level fault signals invisible to Prometheus.

REFERENCES

[1] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *IEEE Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003. DOI: https://doi.org/10.1109/MC.2003.1160055

[2] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, pp. 70–93, Jan.–Feb. 2016. DOI: https://doi.org/10.1145/2898442.2898444

[3] D. Ford, F. Labelle, F. I. Popovici, M. Stokely, V.-A. Truong, L. Barroso, C. Grimes, and S. Quinlan, "Availability in globally distributed storage systems," in *Proc. 9th USENIX Conf. Operating Systems Design and Implementation (OSDI)*, Vancouver, BC, Oct. 2010, pp. 61–74. DOI: https://dl.acm.org/doi/10.5555/1924943.1924948

[4] B. H. C. Cheng *et al.*, "Software engineering for self-adaptive systems: A research roadmap," in *Software Engineering for Self-Adaptive Systems*, ser. Lecture Notes in Computer Science, vol. 5525, Springer, Berlin, 2009, pp. 1–26. DOI: https://doi.org/10.1007/978-3-642-02161-9_1

[5] O. Ibidunmoye, F. Hernandez-Rodriguez, and E. Elmroth, "Performance anomaly detection and bottleneck identification," *ACM Computing Surveys*, vol. 48, no. 1, pp. 1–35, Aug. 2015. DOI: https://doi.org/10.1145/2791120

[6] J. Weng, J. Wang, J. Yang, and Y. Yang, "Root cause analysis of anomalies of multitier services in public clouds," *IEEE/ACM Transactions on Networking*, vol. 26, no. 4, pp. 1646–1659, Aug. 2018. DOI: https://doi.org/10.1109/TNET.2018.2843805

[7] A. Samir and C. Pahl, "Self-managing container deployments in Kubernetes," in *Proc. 9th Int. Conf. Cloud Computing and Services Science (CLOSER)*, Heraklion, Greece, May 2019, pp. 335–342. DOI: https://doi.org/10.5220/0007731803350342

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, Kigali, Rwanda, May 2023. DOI: https://doi.org/10.48550/arXiv.2210.03629

[9] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. 37th Conf. Neural Information Processing Systems (NeurIPS)*, New Orleans, LA, Dec. 2023. DOI: https://doi.org/10.48550/arXiv.2303.11366

[10] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024. DOI: https://doi.org/10.1007/s11704-024-40231-1

[11] T. Guo *et al.*, "Large language model based multi-agents: A survey of progress and challenges," in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, Jeju, South Korea, Aug. 2024. DOI: https://doi.org/10.48550/arXiv.2402.01680

[12] C. Qian *et al.*, "Experiential co-learning of software-developing agents," in *Proc. 62nd Annual Meeting Assoc. Computational Linguistics (ACL)*, Bangkok, Thailand, Aug. 2024. DOI: https://doi.org/10.18653/v1/2024.acl-long.305

[13] M. Ma, J. Xu, Y. Wang, P. Chen, Z. Zhang, and P. Wang, "AutoMap: Diagnose your microservice-based web applications automatically," in *Proc. The Web Conf. 2020 (WWW)*, Taipei, Taiwan, Apr. 2020, pp. 246–258. DOI: https://doi.org/10.1145/3366423.3380111

[14] B. Schroeder, E. Pinheiro, and W.-D. Weber, "DRAM errors in the wild: A large-scale field study," *Communications of the ACM*, vol. 54, no. 2, pp. 100–107, Feb. 2011. DOI: https://doi.org/10.1145/1897816.1897844

[15] M. Levy and Y. Shkuro, "Jaeger: Open source, end-to-end distributed tracing," arXiv preprint arXiv:1903.07574, 2019. DOI: https://doi.org/10.48550/arXiv.1903.07574

[16] J. Romano, J. D. Kromrey, J. Coraggio, and J. Skowronek, "Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys?" in *Proc. Annual Meeting Florida Assoc. Institutional Research*, Cocoa Beach, FL, Feb. 2006. DOI: https://doi.org/10.1080/10705510709336742

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Language Processing (EMNLP)*, Hong Kong, China, Nov. 2019, pp. 3982–3992. DOI: https://doi.org/10.18653/v1/D19-1410

[18] H. Chase *et al.*, "LangChain: Building applications with LLMs through composability," GitHub repository, 2022. DOI: https://doi.org/10.5281/zenodo.10469799

[19] The Kubernetes Authors, "Kubernetes Python client," GitHub repository, 2024. DOI: https://doi.org/10.5281/zenodo.10870064

[20] J. Weng, J. Wang, and Y. Yang, "Understanding and detecting software upgrade failures in distributed systems," in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP)*, Virtual Event, Oct. 2021, pp. 116–131. DOI: https://doi.org/10.1145/3477132.3483577

[21] W. Iqbal, A. N. Toosi, A. Bokhari, and R. Ranjan, "Cost-efficient and SLA-aware microservice deployment in public clouds," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 1127–1140, Mar.–Apr. 2023. DOI: https://doi.org/10.1109/TSC.2022.3165699

[22] S. Tuli, S. Casale, and N. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time-series data," *Proceedings of the VLDB Endowment*, vol. 15, no. 6, pp. 1201–1214, Feb. 2022. DOI: https://doi.org/10.14778/3514221.3514232

[23] C. Luo, J.-G. Lou, Q. Lin, Q. Fu, R. Ding, D. Zhang, and Z. Wang, "MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments," in *Proc. The Web Conf. 2021 (WWW)*, Virtual Event, Ljubljana, Slovenia, Apr. 2021, pp. 3581–3591. DOI: https://doi.org/10.1145/3442381.3449905

[24] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, Mar. 2022. DOI: https://doi.org/10.1145/3501297

[25] D. Balla, C. Simon, and M. Maliosz, "Adapting the scaling of Kubernetes deployments with reinforcement learning," in *Proc. IEEE Int. Conf. Autonomic Computing and Self-Organizing Systems (ACSOS)*, Washington, DC, Aug. 2020, pp. 158–166. DOI: https://doi.org/10.1109/ACSOS49614.2020.00034

[26] J. Lin, X. Zhao, A. Blokhin, and I. Kovalev, "Evolutionary multi-agent reinforcement learning for autonomous network slice management," *IEEE Transactions on Cognitive Communications and Networking*, vol. 9, no. 3, pp. 601–614, Jun. 2023. DOI: https://doi.org/10.1109/TCCN.2023.3237758

[27] W. Zhang *et al.*, "Intelligent operations for cloud native systems via AIOps," *IEEE Transactions on Cloud Computing*, vol. 12, no. 1, pp. 67–82, Jan.–Mar. 2024. DOI: https://doi.org/10.1109/TCC.2023.3271459

[28] S. Deng *et al.*, "An empirical study of deep learning models for vulnerability detection," in *Proc. 45th IEEE/ACM Int. Conf. Software Engineering (ICSE)*, Melbourne, Australia, May 2023. DOI: https://doi.org/10.1109/ICSE48619.2023.00084

[29] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, Kigali, Rwanda, May 2023. DOI: https://doi.org/10.48550/arXiv.2111.09543

[30] T. Sun *et al.*, "Beyond the imitation game: Quantifying and extrapolating the capabilities of language models," *Transactions on Machine Learning Research*, 2023. DOI: https://doi.org/10.48550/arXiv.2206.04615

Baby Sree Gangarapu is an undergraduate student in the Department of Artificial Intelligence at Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, India. Her research interests encompass autonomic cloud computing, multi-agent reasoning systems, Kubernetes orchestration, and self-healing distributed architectures.

Hindupur Raghavender Vardhan is an undergraduate student in the Department of Artificial Intelligence at Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, India. His research focuses on cloud-native observability, distributed telemetry streaming, time-series anomaly detection, and automated remediation controllers.

Yeddula Likitha Reddy is an undergraduate student in the Department of Artificial Intelligence at Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, India. Her research interests include AIOps, non-parametric statistical benchmarking of distributed systems, and policy-constrained agent execution environments.

Chintakunta Sai Jagruth is an undergraduate student in the Department of Artificial Intelligence at Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, India. His research work centers on container runtime isolation, kernel cgroups mechanics, network fault injection, and microservice resilience engineering.